

Experimenting with Retrieval Augmented Generation

Contents

1. Preface	1
2. Introduction	1
3. Dataset	1
4. Implementation Steps	2
4.1. Chunking	2
4.2. Semantic Search and Vector Embeddings	3
4.3. Retrieve and Rerank the Results	3
4.4. Additional Retrieval Enhancements	4
4.5. Generate the Answer	4
5. Technical Implementation Details	5
6. Results	5
7. Project Limitations	6
8. Security	6
9. Structural Limitations and the Future of RAG	7
9.1. The Rise of Agentic RAG	7
10. Conclusions	8

1. Preface

The purpose of this project is to gain both a solid theoretical understanding and practical, hands-on experience with the Retrieval Augmented Generation (RAG) technology. Through this work, the project aims to explore the core concepts underlying RAG, and also all the technical nuances that have arisen during the implementation.

2. Introduction

Retrieval Augmented generation (RAG) is a method that allows to extend an LLMs internal knowledge with domain specific knowledge not available in the training data. For example this could allow an LLM to comprehend internal company information and generate an appropriate response based on the retrieved content.

RAG emerged as an elegant solution borrowed directly from search engines. Just as Google displays links with relevant snippets for your query, RAG retrieves the most pertinent document fragments and feeds them to the LLM for synthesis. The core idea is beautifully simple: if you can't fit everything in context, find the most relevant pieces and use those. It turns LLMs into sophisticated search result summarizers.

3. Dataset

The example dataset used to evaluate the project is the **LegalBench-RAG** dataset. It contains a series of legal documents as .txt files, they belong to different categories such as **Non Disclosure Agreements**, **Privacy Policies**, and other legal documents such as **Consultancies** or **Merges**, for a total of 698 files. Because of limited disk resources and a limited amount of free LLM tokens

available only up to 10 random files for each category have been picked. The dataset also contains a series of question-answer pairs for each file, where the answer is a citation of the file where the exact answer can be found.

It has been decided to not take into consideration other datasets, for example documents including tables, numbers, financial informations as PDF formats have been left out due to their excessive complexity and pitfalls which weren't really suitable for an introductory project.

4. Implementation Steps

In order to implement Retrieval Augmented Generation there are many steps that we need to take care, all of which are very important, neglecting one of the could effect the quality of our LLM answer.

4.1. Chunking

Chunking is the practice of breaking documents into manageable pieces that can be embedded, stored, and retrieved. We can think of chunking as the practice of splitting a large text into paragraphs so the model can focus on understanding only the snippets that matters instead of an entire book. In this way the model sees the text that is directly relevant and eliminates the irrelevant noise around it. There is also a practical limit, LLMs and embeddings have a context limit, so they can process a limited number of tokens (tokens are small pieces of text). It is fairly easy for a computer to break up a text stored as a simple string. But when dealing with complex document formats such as PDFs, Word Documents or Powerpoints or even Webpages this task starts to become quite difficult. Our goal is to process these documents and convert them to a machine-processable format. One popular tool is **Docling**, an open-source, very capable and efficient document conversion tool. **Docling** uses specialized AI models for layout analysis and table structure recognition. Optionally it can also leverage OCR engines for analyzing scanned PDFs. The final result can be presented as structured formats like Markdown, HTML or JSON, which are usually splitted by paragraph.

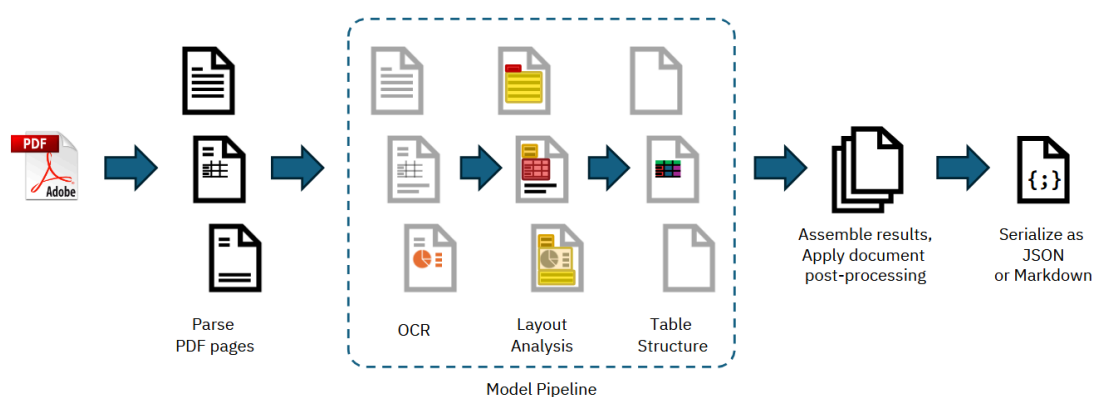


Figure 1: A Sketch of Docling's default processing pipeline. The inner part of the model pipeline is easily customizable and extensible

Although complex documents weren't in the scope of the project it has been decided to implement an endpoint for testing purposes. In this way it was possible to understand how chunking works for structured records, in particular it was useful for acquiring experience with the Docling library

4.2. Semantic Search and Vector Embeddings

Retrieval is a key component of the RAG process. As we've seen before we have obtained our document chunks, now we need a way to efficiently retrieve them if they are related to our answer. In order to do so we need to find a method to encapsulate the meaning of a piece of text into a value and retrieve the snippets whose value is similar to our value. This is the idea behind what is called “**Semantic Search**”. We feed our text input to a Deep Learning model which gives us a vector of fixed length (a list of real numbers). This vector encodes the semantic meaning of the text, so geometrically two phrases that are semantically similar are represented as two vectors that are close in the vector space (that is the distance between them is low).

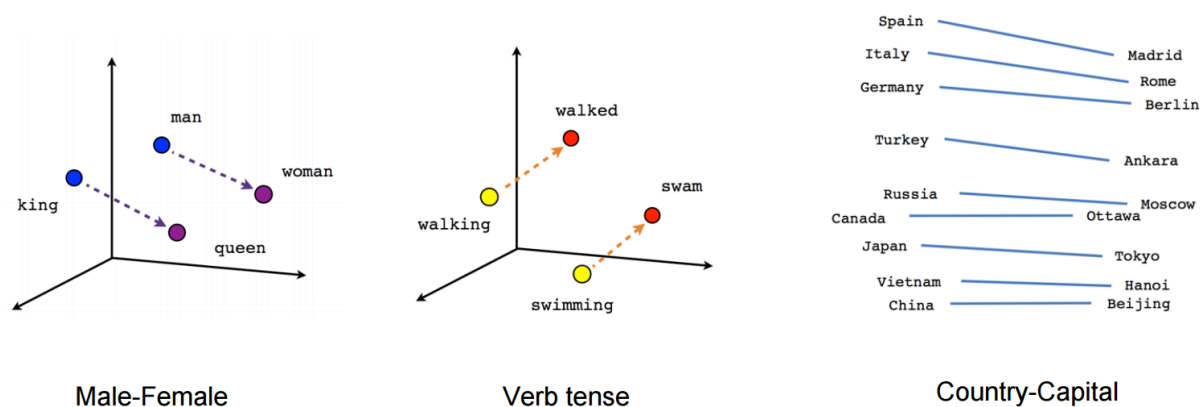


Figure 2: How Words are represented in three and two dimensional spaces

As we can see in Figure 2 the words “Walking” and “Walked” are close, since their meaning is very similar. It is also important to notice that other useful geometric properties arise. Notice how the distance between “King” and “Queen” is similar to the distance between “Man” and “Woman”. Our Deep Learning model is able to fully encapsulate all the different facets surrounding the semantic aspects of a word. But of course this can be easily applied to sentences as well.

Modern embeddings models such as BERT, RoBERTa, and sentence transformers are usually based on a **transformer architecture** (notice how LLMs are also based on a transformer architecture). The model is trained using a **contrastive learning** approach. The architecture's learning is based on pairs (or groups), the model is rewarded for generating two geometrically close vectors given two similar sentences (and punished if this is not the case). And at the same time the model is rewarded for generating two distant vectors given two unrelated sentences. In this way the model learns to pull similar sentences closer together in the embedding space, and push dissimilar sentences farther apart, and thus learning to effectively map the semantic meaning of a sentence into a vector space.

For this project it has been used the “all-MiniLM-L6-v2” model of the sentence transformers family from [Hugging Face](#)

4.3. Retrieve and Rerank the Results

So far we have transformed our texts into vectors, placed them all into a vector space, and compared their proximity to a query vector using a distance metric. But we may have noticed that our top k , while being relevant to our answer, are not the most relevant ones we could have obtained. This is because it is difficult to perfectly represent a paragraph into a vector without some amount of information loss. This is where we introduce an additional step, a **reranking model**, a type of model that, given a query and document pair, will output a similarity score. We will use this score to reorder the documents by relevance to our query.

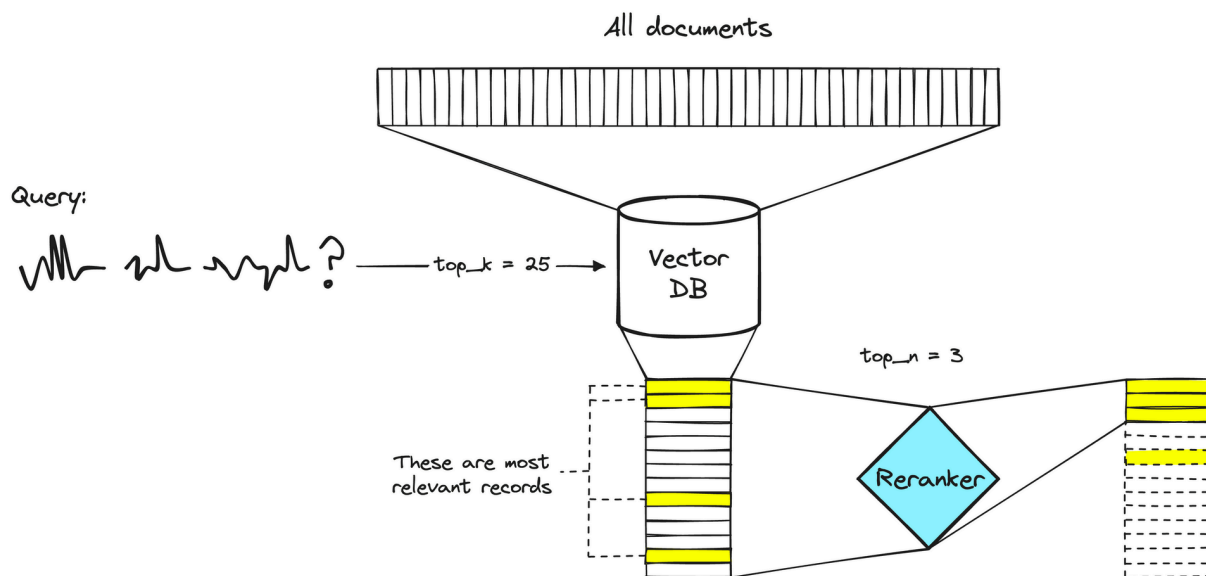


Figure 3: Representation of the two-stage retrieval system

Another reason for this loss of accuracy is that encoders have no context on the query because we don't know the query until we receive it (we create embeddings before user query time). On the other hand, a reranker can receive the raw information directly into the large transformer computation, meaning less information loss. Because we are running the reranker at user query time, we have the added benefit of analyzing our document's meaning specific to the user query, rather than trying to produce a semantic vector and then retrieving similar semantic vectors. The actual model used in our implementation is called [FlashRank](#).

4.4. Additional Retrieval Enhancements

A purely vector based retrieval implementation performs quite poorly. For instance the embedding model finds quite difficult to understand mentions of specific companies or terms. For this reason we have adopted a **Hybrid Retrieval** approach. The vector retriever is combined with a **Full Text Search** Retriever. The **BM25 Function**, a retrieval function that excels at exact term matching, unlike embeddings. These are combined with the results obtained with the vector search and then feed to the Reranker.

Additionally, it has been decided to inject a short LLM-generated summary of the document as a metadata for each document chunk. This has been shown to improve the quality of the retrieved results by quite a lot. Plus it also helps the generation of the final answer by the LLM by providing additional context.

4.5. Generate the Answer

Our LLM now has the necessary context to answer our question. At this step our chunks are feeded to the LLM through **prompt engineering**, specifically by inserting them into the prompt's context. Some chunks may not be relevant to the question but the LLM is still able to pick the right ones for the final answer. In practice, the retrieved chunks are concatenated with the user's query into a single prompt that follows this general structure:

Context: [Retrieved Chunk 1] [Retrieved Chunk 2] [Retrieved Chunk 3]

Question: (The topic of the conversation is [A short summary of the user's previous questions on this conversation]). [User's original question]

Answer based on the context above:

In addition, it also has been added the capability of remembering the context of the conversation. Previous questions are fed to an LLM and a short summary of the conversation topic is generated, and attached to the final prompt as described above. *(If there are no messages this part is ignored)*

Regarding the LLM model, `mistral-large-latest` from **Mistral**, a young but innovative French tech company, is used for every step.

5. Technical Implementation Details

The tools used for this project have been picked in order to deliver a modern and understandable implementation. For the web server it has been decided to use **FastAPI**, a simple web framework for the **Python** programming language. It's able to deliver REST APIs as well as HTML pages, using the **Jinja** template engine. For storing user's data and messages **SQLite** is used as storage, but an external one could be easily integrated with just a few lines of code.

For the logic behind all the operations described in the previous sections it has been used **LangChain**. It is an open source library designed to simplify the development of applications powered by large language models. The most useful advantage of the library is that it provides a handy modular architecture which includes all the specific models described before. All these useful abstraction helped with keeping the code clean and concise and avoid any boilerplate.

6. Results

The chatbot is able to comprehend the questions and provide an answer similar to those presented in the benchmarks, meaning that the retrieval phase is working as expected. Moreover when provided several distinct questions in a row the bot is able to answer correctly even if only the first one mentions the company name, this suggests that the context of the conversation is actually understood. As a second check and also for debugging purposes every step of the pipeline is logged, so by manually checking in the logs we have noticed a correct context and several relevant chunk of texts in the first positions of the retrieved list.

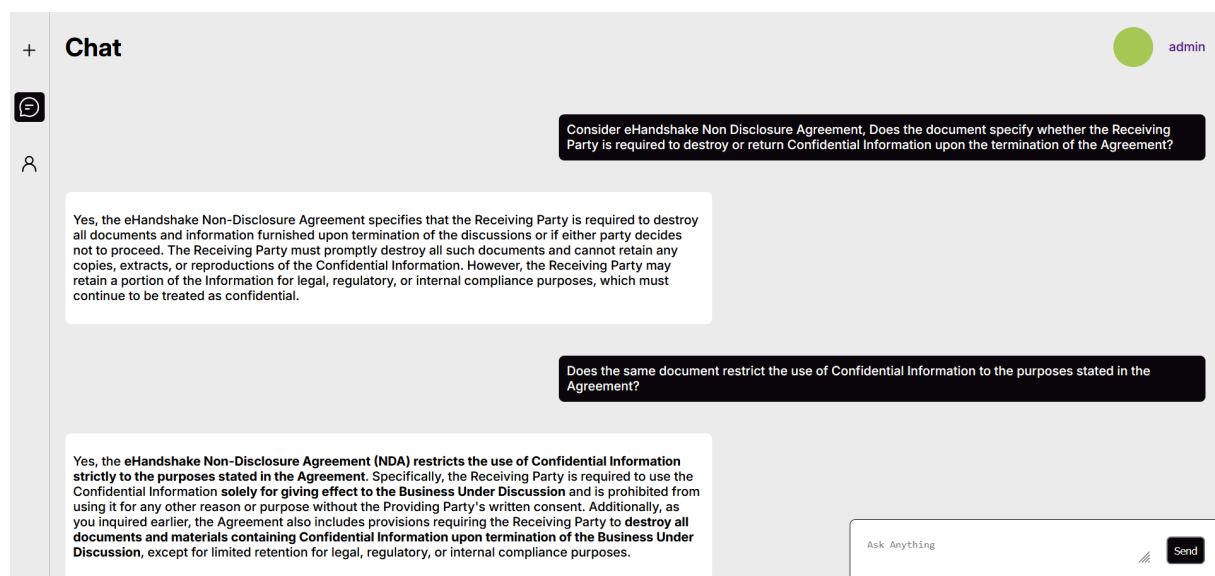


Figure 4: An example of a conversation regarding a Non Disclosure Agreement

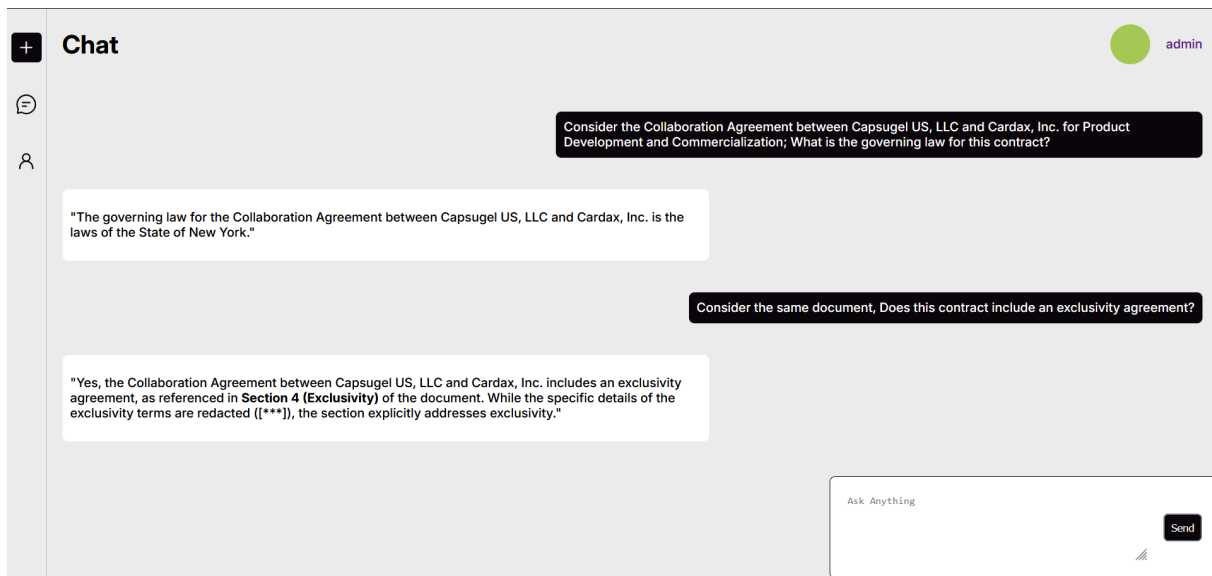


Figure 5: Another example, this time the topic is a Collaboration Agreement

7. Project Limitations

Because this is an amateur project, several limitations have been noticed. The first one concerns the chunking strategy, optimistically the best chunks are tailored around the data (in this case legal documents) and also each chunk should be a “semantic section” of the text and not just a list of characters of length n . Another limit concerns document retrieval and is very specific. For example while composing the question, if the company is mentioned like this: Regarding LVMH's agreement or this Regarding the company "LVMH" will produce low quality results. The entity should be simply named like this I want to ask about the LVMH agreement, without apostrophes, quotes or any character in between. We suspect this is a limitation of the FTS retriever, that could be easily solved.

In addition to this, most of the ML models and tools revolving around this project are open-source (except for the LLM model). While this is most of the times a benefit for certain cases (for example if we have the necessity to implement an air-gapped solution) some enterprise products could perform better.

One particular aspect not covered is **Query Expansion** and **Query Rewriting**. The first method consist in rewriting the user's question with the aid of an LLM, into a more cleaner or detailed version, in order to improve the quality of the retrieved documents. The second method involves generating several paraphrased versions of the query, then after retrieving the results for each they are merged together without any duplicates, subsequently the pipeline proceeds as usual.

Another future improvements could also be an automatic testing workflow. Due to the datasets already having question answer pairs for each text, an LLM or another algorithm could evaluate the accuracy and pertinence of each answer autonomously. Due to the limited quantity of tokens available this methodology was not adopted.

8. Security

The rapid raise in popularity of Artificial Intelligence has brought a new wave of security challenges that companies have few experience with. The **lethal trifecta** (sometimes called the “lethal trifecta attack”) is a term coined in 2025 by AI researcher and developer Simon Willison. It describes a

dangerous security vulnerability that concerns LLM based systems. When a platform has all three of the following characteristics it becomes vulnerable to data theft **indirect prompt injection** (hidden malicious instructions embedded in content the AI processes). **Access to private sensitive data** is the first aspect, this involves the ability to read emails, documents, databases, files, source code, or other confidential information. The second is **Exposure to untrusted content**, an user could submit questions, documents, emails, etc. This content can contain hidden instructions that the AI interprets as commands. The last characteristic is the **Ability to externally communicate**, the system can send data outward, for example by making web requests, sending emails, posting to public channels, calling APIs, or writing to external systems. An attacker could then trick the AI into following hidden instructions, these consists of retrieving the private data and exfiltrating it through an external communication channel. LLM-level mitigations such as encoding guardrails in the prompt or using AI scanning to detect potential attacks may not work 100% of the times as these measures could be circumvented. The most reliable solution is to remove or strictly restring one of the three key characteristics of the attack. For instance the retrieval tool could also accept the user's read permissions as input at only retrieve the minimal data it needs for correctly completing the task and comply to the user's privilege levels.

9. Structural Limitations and the Future of RAG

One evident limitation of the RAG approach is the fragility of every single step. Each stage if not implemented correctly can add some cascading error effects that reflects on a poor quality final result. The complexity of maintaining such elaborated system can also elevate the cost of maintaining the overall program.

Other structural limitations come from the documents themselves:

1. **Context Fragmentation**, long documents are interconnected webs and not independent paragraphs, so a single question might require information from many different documents. Chunking is not able to preserve these relationships
2. **Semantic Search Fails on Numbers**, our dataset was not overly complex and didn't contain overly structured documents with lots of numbers. However in general is important to notice that "\$45.2M" and "\$45,200,000" may have different embeddings, semantic search is not very good with numbers.
3. **No Causal Understanding**, RAG can't follow "See Note 12" → Look up and retrieve Note 12
4. **The Vocabulary Mismatch Problem**, Companies can use different terms for the same concept ("Adjusted EBITDA" vs "Operating Income Before Special Items")
5. **Temporal Blindness**, RAG can't grasp dates and times (example: can't distinguish Q3 2024 from Q3 2023 reliably)

9.1. The Rise of Agentic RAG

In May 2025, Anthropic released Claude Code, an AI coding agent that works in the terminal. What immediately stood out was it's ability to use filesystem tools like Grep (a tools that allows to search for keywords and patterns inside files) and Glob (a tool that allows to search files by extension and date, and other stuff). In summary an Agent doesn't simply retrieve, it can investigate, understand incrementally and correct itself based on findings. In conclusion, it can effectively do what RAG is supposed to do but without RAG technology.

It's also important to notice that the cost of processing text with LLMs is decreasing, and their power and accuracy is increasing, especially their context window. The **Context Window** of an

LLM is the maximum amount of text the model can read and consider at one time when generating a response. Here are some numbers:

2022-2025:

- GPT-4: 8K tokens (12 pages)
- GPT-4-32k: 32K tokens (50 pages)

2025:

- Claude Sonnet 4: 200k tokens (700 pages)
- Gemini 2.5: 1M tokens (3,000 pages)
- Grok 4-fast: 2M tokens (6,000 pages)

We can notice how entire documents can be fitted into the entire context window now, LLMs are able to reason on the full extent of huge documents, instead of snippets of text retrieved from an extremely engineered retrieval tool.

10. Conclusions

This experience has been extremely useful for me. I was able to learn how Retrieval Augmented Generation works, both from a theoretical and practical point of view. In particular, this project allowed me to learn all the nuances regarding the implementation of the RAG technology, such as which tactics can improve the quality and relevance of the retrieved documents. I was also able to understand the direction to where the industry is moving, and how newer solutions are able to tackle the same problems as RAG but many times more accurately. It can be said that the project has accomplished the task it set out to achieve, namely to convey a comprehensive, 360 degree understanding of RAG technology and the challenges it can address.